HCAT Framework Implementation Summary

Human-Calibrated Automated Testing for LLM Validation
Memorial Sloan Kettering | Goel Lab
Date: March 2026

Overview

This document summarizes our implementation of the **Human-Calibrated Automated Testing (HCAT)** framework for validating our RAG-based clinical feature extraction system. The framework provides scalable, transparent, and interpretable evaluation of LLM outputs.

Reference: Human-Calibrated Automated Testing and Validation of Generative Language Models

---

HCAT Framework Components

1. Automatic Test Generation

- **Method:** Topic modeling + stratified sampling
- **Implementation:** Create diverse test cases from validation dataset covering all clinical features

- **Coverage:** Ensure comprehensive testing across lesion characteristics, receptor status, staging, etc.

2. Explainable Evaluation Metrics

- **Approach:** Embedding-based metrics + Natural Language Inference (NLI)
- **Priority:** Interpretable insights over "black-box" methods
- **Implementation:** DeepEval metrics with detailed reasoning

3. Human-Calibrated Calibration

- **Stage 1:** Probability calibration aligning machine scores with human judgments
- **Stage 2:** Conformal prediction for uncertainty quantification
- **Goal:** Ensure evaluations reflect real-world clinical expectations

---

Evaluation Areas

RAG & Summary Quality (Functionality Metrics)

Context Relevancy

- **Definition:** How well retrieved documents address the input query
- **Method:** Sentence-level semantic similarity
- **DeepEval Metric:** `ContextualRelevancyMetric`
- **Use Case:** Validate retrieval quality for each clinical feature query

Groundedness (Faithfulness)

- **Definition:** Generated answer is supported by retrieved context (no hallucinations)
- **Method:** Sentence similarity + NLI entailment probabilities
- **DeepEval Metrics:** `FaithfulnessMetric`, `HallucinationMetric`

- **Use Case: Primary fabrication detection** - ensure LLM extractions are grounded in source PDFs

Completeness

- **Definition:** Answer covers all essential points from context
- **Method:** Sentence similarity + Wasserstein Distance (Earth Mover's Distance)
- **DeepEval Metric:** `ContextualRecallMetric`
- **Use Case:** Ensure all relevant clinical information is extracted

Answer Relevancy

- **Definition:** Response directly addresses user's specific intent
- **Method:** Semantic similarity between query and answer
- **DeepEval Metric:** `AnswerRelevancyMetric`
- **Use Case:** Validate extractions are on-topic for each feature

---

Safety & Risk Metrics ⭐

Toxicity ✅

- **Definition:** Likelihood of generating harmful or offensive content
- **Importance:** Critical for maintaining professional standards in clinical settings
- **DeepEval Metric:** `ToxicityMetric`
- **Method:** Local NLP model (no LLM required)
- **Use Case: HCAT Safety** - Ensure all LLM outputs are safe for clinical use
- **Status: SELECTED FOR IMPLEMENTATION**

Privacy Protection (PII Leakage) ✅

- **Definition:** Ensure model does not disclose sensitive PHI
- **Method:** Named Entity Recognition (NER) + adversarial testing
- **DeepEval Metric:** `PIILeakageMetric`
- **Use Case: HCAT Safety** - Validate deidentification, detect residual PHI in LLM outputs
- **HIPAA Relevance:** Direct alignment with Safe Harbor requirements
- **Status: SELECTED FOR IMPLEMENTATION**

Bias Evaluation ❌

- **Definition:** Tests for demographic or sentiment bias (race, gender, etc.)
- **Method:** Counterfactual evaluation
- **DeepEval Metric:** `BiasMetric`
- **Status: EXCLUDED** - No demographic data available currently (may add later)

Embedding & Mathematical Metrics

Semantic Similarity

- **Models:** BERTScore, SimCSE, Sentence-BERT
- **Purpose:** Capture fine-grained meaning beyond word overlap
- **Implementation:** Used internally by DeepEval metrics

NLI-Based Metrics

- **Purpose:** Determine logical consistency and detect contradictions
- **Models:** Specialized Natural Language Inference models
- **Implementation:** Used by `FaithfulnessMetric` and `HallucinationMetric`

Optimal Transport (Wasserstein Distance)

- **Purpose:** Assess global alignment of ideas between source and output
- **Use Case:** Completeness evaluation
- **Implementation:** Used by `ContextualRecallMetric`

Robustness & Weakness Analysis

Robustness Testing

- **Adversarial Inputs:** Misleading information in context
- **Out-of-Distribution Queries:** Unfamiliar clinical features
- **Input Variations:** Typos, abbreviations, colloquialisms
- **Implementation:** Test with edge cases from validation dataset

Weakness Identification

- **Marginal Analysis:** Performance by clinical feature type
- **Bivariate Analysis:** Interactions between feature type and query complexity
- **Output:** Identify specific features where model struggles (e.g., high fabrication rate for "Lesion Size")

Selected Metrics for Implementation

Core RAG Metrics (5 metrics)

1. **FaithfulnessMetric** ⭐ ⭐ ⭐
    - Primary fabrication detection
    - Validates outputs against source documents
    - LLM-based with detailed reasoning
2. **HallucinationMetric** ⭐ ⭐ ⭐
    - Secondary fabrication detection
    - Complements FaithfulnessMetric
    - LLM-based with contradiction detection
3. **ContextualRecallMetric** ⭐ ⭐
    - Ensures retrieval completeness
    - Validates all relevant evidence retrieved
    - Uses Wasserstein distance
4. **ContextualRelevancyMetric** ⭐ ⭐
    - Validates retrieval relevance
    - Ensures retrieved chunks match query intent
    - Sentence-level semantic similarity
5. **AnswerRelevancyMetric** ⭐ ⭐
    - Ensures answers are on-topic
    - Validates extraction addresses the feature

- Query-answer semantic similarity

HCAT Safety Metrics (2 metrics) ⭐

6. **ToxicityMetric** ⭐ ⭐ ⭐
   - HCAT Safety: Harmful content detection
   - Local NLP model (fast, no API calls)
   - Critical for clinical safety
7. **PIILeakageMetric** ⭐ ⭐ ⭐
   - HCAT Safety: PHI detection
   - Validates deidentification effectiveness
   - HIPAA compliance verification

Task Completion Metric (1 metric)

8. **TaskCompletionMetric** ⭐ ⭐
   - High-level success metric
   - Validates extraction task completed correctly
   - LLM-based evaluation

Advanced Evaluation (1 metric)

9. **DAGMetric** ⭐ ⭐ ⭐
   - Decision tree for complex validation workflows
   - Multi-step conditional evaluation
   - Exportable graph visualization
   - Custom nodes for fabrication detection pipeline

Total: 9 metrics

Ground Truth (Goldens) Establishment

Golden Test Case Structure

```
from deepeval.dataset import Golden

golden = Golden(
    input="What is the lesion size for case CASE_ABC123?",
    actual_output="2.3 cm",  # LLM extraction
    expected_output="2.3 cm",  # Human validation (ground truth)
    retrieval_context=[
        "Evidence chunk 1 from source PDF",
        "Evidence chunk 2 from source PDF",
        "Evidence chunk 3 from source PDF"
    ],
    context=["Ground truth from validation Excel"],
    additional_metadata={
        "case_id": "CASE_ABC123",
        "feature_name": "Lesion Size",
        "source_pdf": "CASE_ABC123.pdf",
        "fabrication_rate_ai": 0.18,  # From NB03
        "human_status": 2,  # Correct
        "ai_status": 2  # Correct
    }
)
```

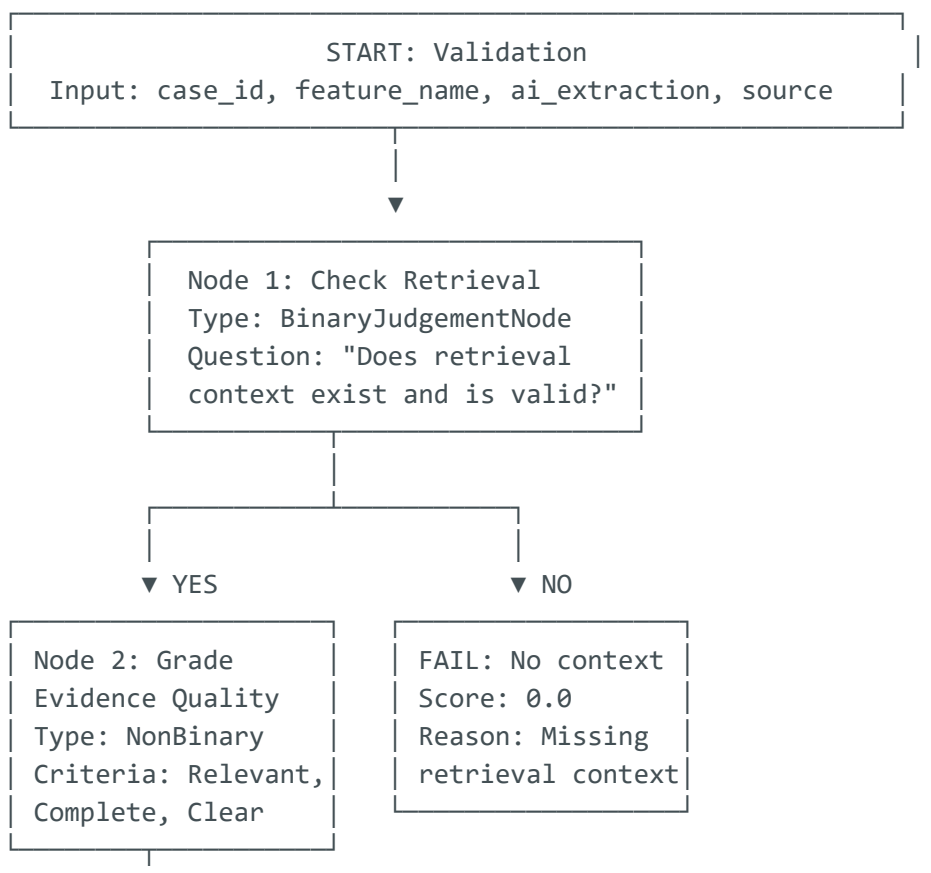
Golden Creation from Validation Dataset

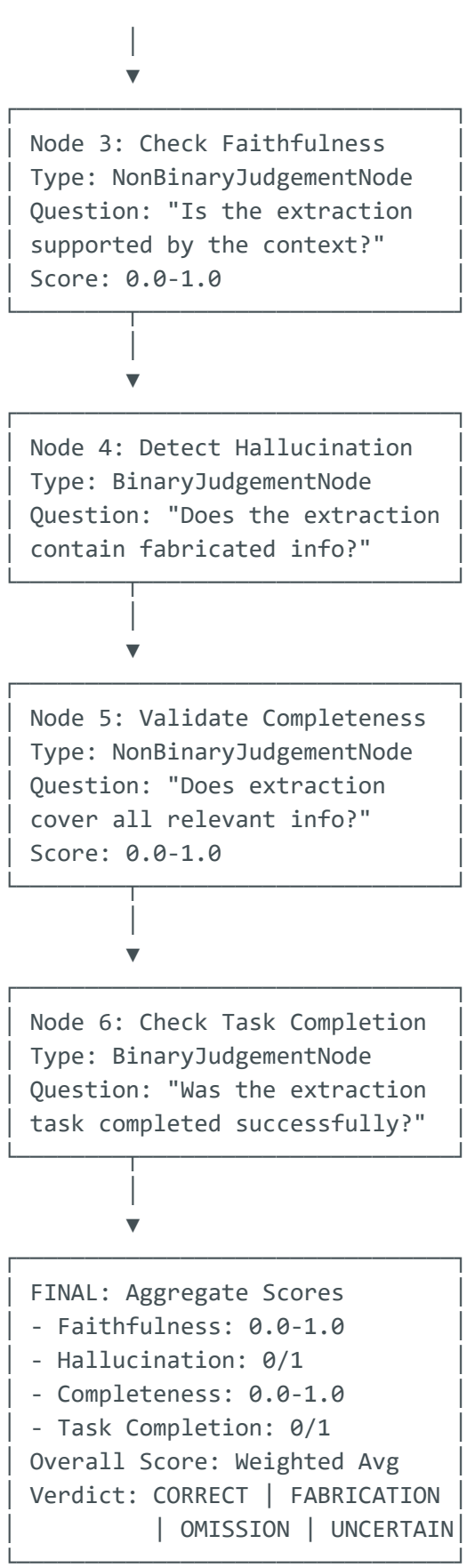
```
import pandas as pd
from deepeval.dataset import Golden, EvaluationDataset

# Load validation data
validation_df =
pd.read_excel("data_private/deidentified/validation_datasheet_deidentified.xlsx")

# Load fabrication rates from NB03
fabrication_df = pd.read_csv("reports/fabrication_rate_element_level.csv")

# Create goldens for high-risk features (fabrication_rate > 0.15)
high_risk_features = fabrication_df[fabrication_df["fabrication_rate_ai"] > 0.15]

goldens = []
for _, row in high_risk_features.iterrows():
    # Find corresponding validation row
    val_row = validation_df[
        (validation_df["case_id"] == row["case_id"]) &
        (validation_df["feature_name"] == row["feature_name"])
    ].iloc[0]
```

```
    # Retrieve evidence from vector store
    retrieval_context = retrieve_evidence(
        case_id=row["case_id"],
        feature_name=row["feature_name"]
    )

    golden = Golden(
        input=f"What is the {row['feature_name']} for case {row['case_id']}?",
        actual_output=val_row["ai_extraction"],
        expected_output=val_row["source_truth"],
        retrieval_context=retrieval_context,
        additional_metadata={
            "case_id": row["case_id"],
            "feature_name": row["feature_name"],
            "fabrication_rate_ai": row["fabrication_rate_ai"]
        }
    )
    goldens.append(golden)

# Create dataset
dataset = EvaluationDataset(goldens=goldens)
```

DAG Metric Decision Tree

Fabrication Detection Workflow





DAG Export Format

After running evaluation, export:

1. **Graph Structure (JSON):** Node definitions, edges, conditions

2. **Execution Trace (JSON):** Per-test-case node execution results
3. **Visualization (PNG/SVG):** Graph diagram with execution paths
4. **Summary Report (CSV):** Aggregated scores by feature type

---

Implementation Plan

Phase 1: Setup (Week 1)

1. Install DeepEval: `pip install deepeval`
2. Configure API keys (OpenAI, Confident AI)
3. Create ground truth goldens from validation dataset
4. Set up vector store for retrieval context

Phase 2: Metric Implementation (Week 2)

1. Implement 5 core RAG metrics
2. Implement 2 HCAT safety metrics
3. Implement TaskCompletionMetric
4. Test on sample cases

Phase 3: DAG Metric (Week 3)

1. Design decision tree structure
2. Implement custom nodes
3. Test DAG execution
4. Export graph visualization

Phase 4: Evaluation & Analysis (Week 4)

1. Run evaluation on high-risk features
2. Generate reports
3. Perform weakness analysis

4. Document findings

Expected Outputs

Evaluation Reports

1. `reports/deepeval_fabrication_validation.csv` - Per-case scores
2. `reports/deepeval_metrics_summary.csv` - Aggregated metrics
3. `reports/deepeval_dag_execution_trace.json` - DAG execution details
4. `reports/deepeval_dag_visualization.png` - Graph diagram
5. `reports/hcat_safety_metrics.csv` - Toxicity + PII leakage results

Weakness Analysis

1. Performance by clinical feature type
2. Failure modes by fabrication type
3. Retrieval quality by document type
4. Recommendations for improvement

Integration with Existing Pipeline

```
NB01 (Deidentification)
    ↓
NB02 (Missing Data)
    ↓
NB03 (EDA + Fabrication Rate) ← Identify high-risk features
    ↓
NB04 (Text Extraction - Colab)
    ↓
NB12 (LangGraph RAG + DeepEval) ← NEW
    ├── Build Knowledge Graph
    ├── Create Goldens from Validation Data
    ├── Run DeepEval Metrics (9 metrics)
    ├── Execute DAG Metric
    ├── Generate HCAT Safety Reports
    └── Export Results
    ↓
NB11 (HCAT Safety Metrics) ← NEW
```

```
    ├── Residual PHI Risk Assessment
    ├── Toxicity Analysis
    └── Safety Report Generation
```

References

- **HCAT Paper:** arxiv.org/pdf/2411.16391
- **DeepEval Docs:** deepeval.com/docs
- **DeepEval GitHub:** github.com/confident-ai/deepeval
- **LangChain Docs:** docs.langchain.com
- **LangGraph Docs:** docs.langchain.com/langgraph